



1 8 0 3

INFORMÁTICA II

Bioingeniería



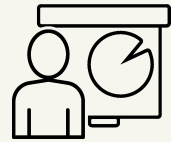
Índice



Para adelantar..



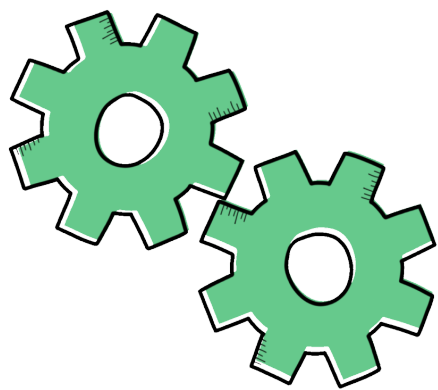
Repaso y dudas



Trabajo en clase



Probelmas adicionales





UNIVERSIDAD
DE ANTIOQUIA

Facultad de Ingeniería

Para adelantar...

3er ejercicio de abstracción



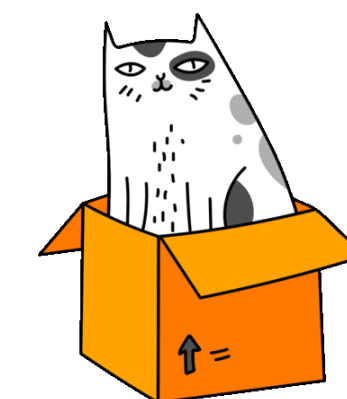
Tercer ejercicio de abstracción

Una clínica veterinaria solicita construir un sistema que permita almacenar datos de las mascotas que tiene hospitalizadas.

Los mascotas tienen información de nombre, número de historia clínica (único para cada mascota), tipo (canino o felino), peso, fecha de ingreso y medicamento que se le está administrando

El sistema debe permitir, opciones:

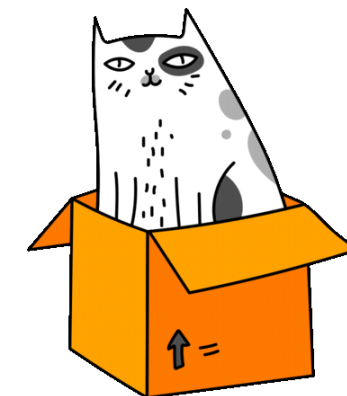
1. Ingresar una mascota al servicio: No debe haber otra mascota con el mismo número de historia clínica. El servicio de hospitalización solo admite 10 pacientes al tiempo
2. Ver fecha de ingreso de una mascota
3. Ver número de mascotas en el servicio
4. Ver medicamento que se está administrando a una mascota
5. Eliminar una mascota del servicio
6. Salir





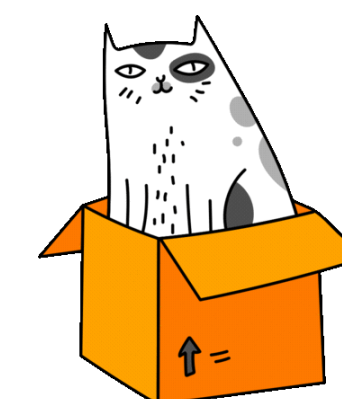
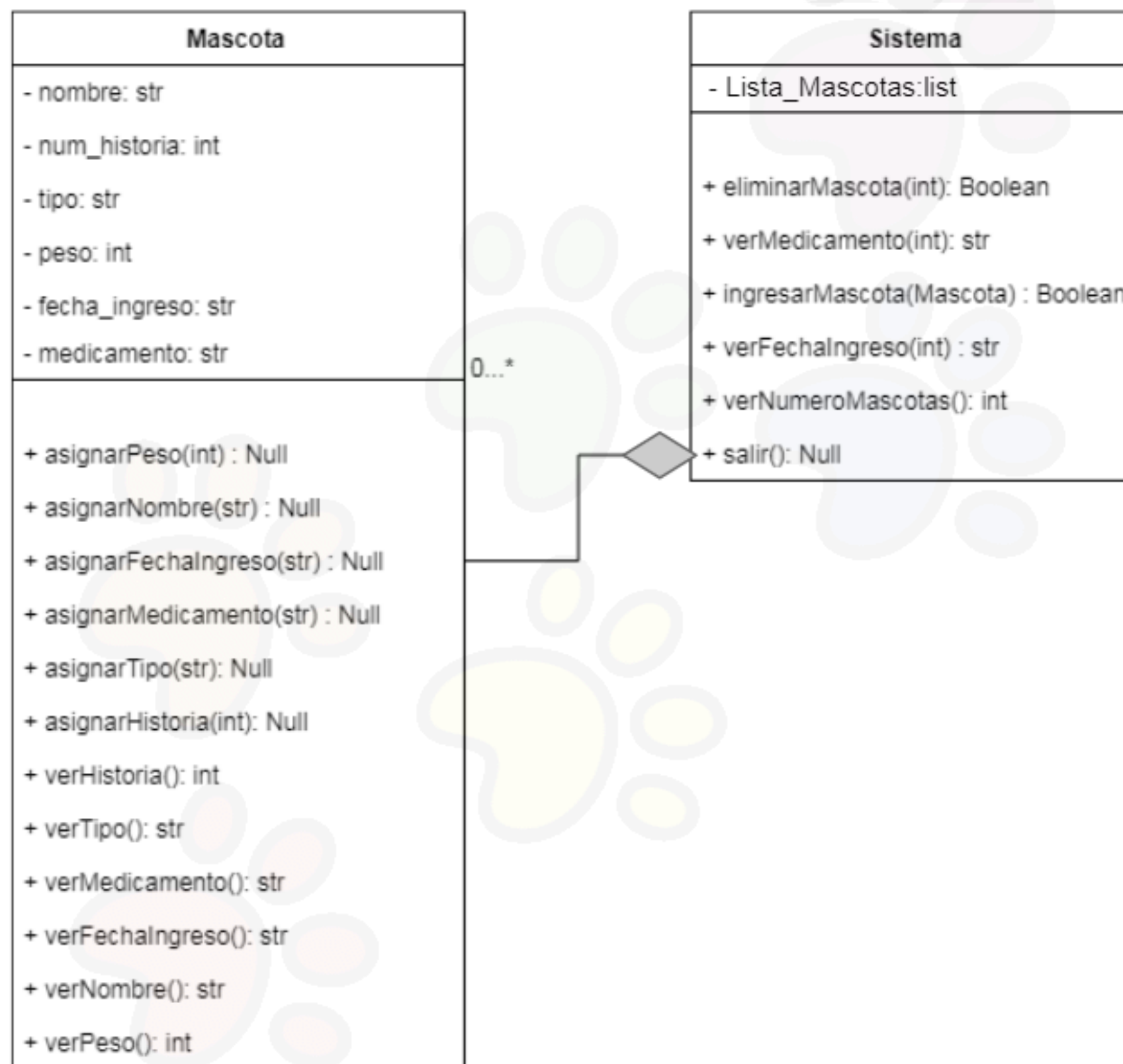
Tercer ejercicio de abstracción

Aplicar los tips mencionados en clase anteriores , abstraer clases, atributos y metodos y realizar diagrama UML, definiendo tipo de relación existente





Tercer ejercicio de abstracción, diagrama UML





**UNIVERSIDAD
DE ANTIOQUIA**

Facultad de Ingeniería

Repaso



Repaso

- Por milunavez 😊, dónde y en que momento existen las variables, los atributos y objetos?
- Que es una clase, que es un objeto?
- Qué es el método main y para que se usa?
- Cómo se crea un repositorio remoto con VS desde la maquina local ?
- Cómo se pública, cómo se trae la información desde Github?
- Qué es un fork ?
- Implementación de contenedores en en clases sistema, cómo seria con una BD real?





**UNIVERSIDAD
DE ANTIOQUIA**

Facultad de Ingeniería

Trabajo en clase



Cuarto ejercicio de abstracción, Version1

Una clínica veterinaria solicita construir un sistema que permita almacenar datos de las mascotas que tiene hospitalizadas.

Los mascotas tienen información de nombre, número de historia clínica (único para cada mascota), tipo (canino o felino), peso, fecha de ingreso y medicamentos que se le está administrando.

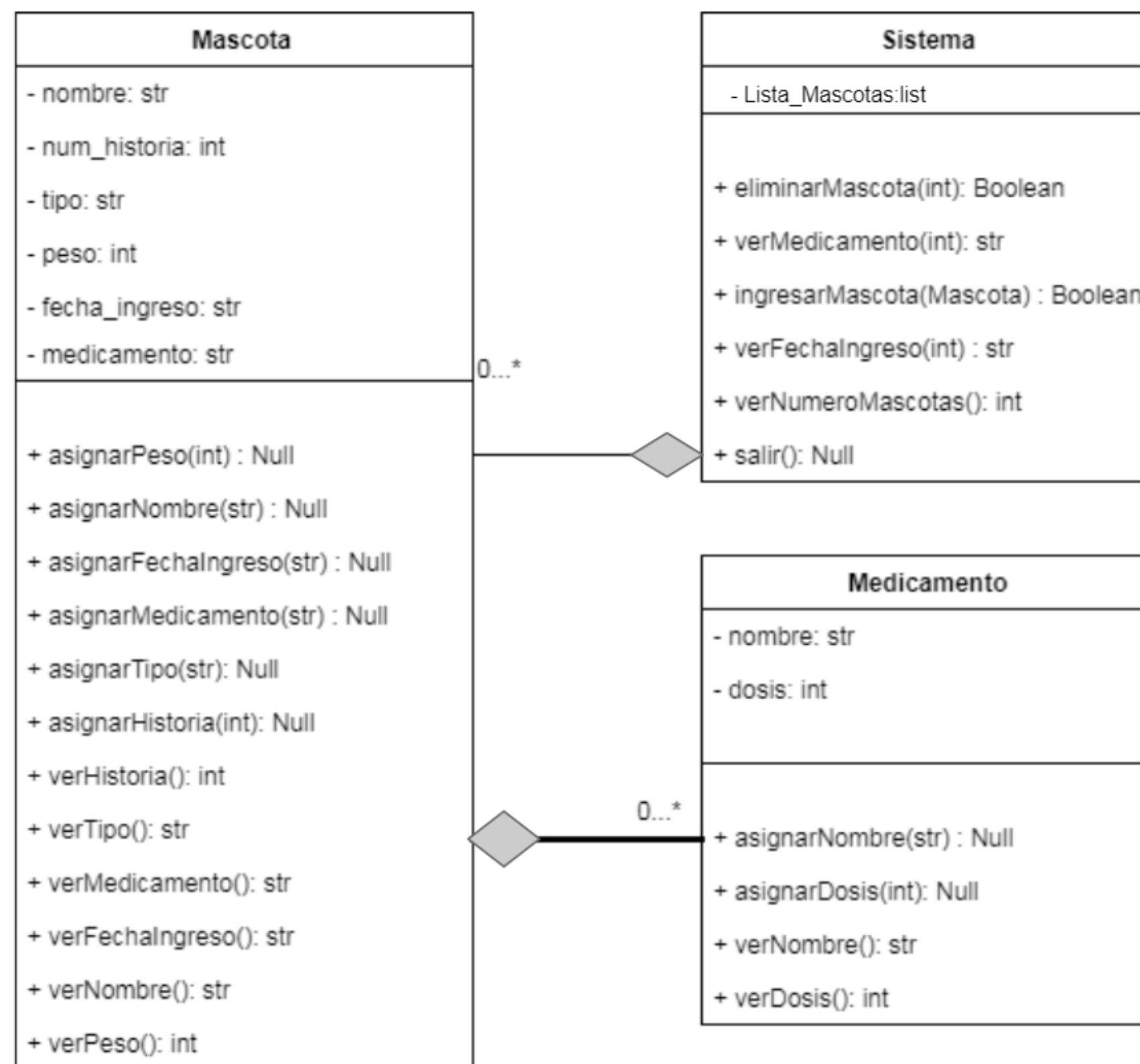
El medicamento tiene información de nombre y dosis

El sistema debe permitir tres opciones:

1. Ingresar una mascota al servicio: No debe haber otra mascota con el mismo número de historia clínica.
El servicio de hospitalización solo admite 10 pacientes al tiempo
2. Ver fecha de ingreso de una mascota
3. Ver número de mascotas en el servicio
4. Ver medicamentos que se está administrando a una mascota
5. Eliminar una mascota del servicio
6. Salir



Cuarto ejercicio de abstracción, UML





Cuarto ejercicio de abstracción, implementación en python

```
class Medicamento:
```

```
    def __init__(self):  
        self.__nombre = ""  
        self.__dosis = 0
```

```
    def verNombre(self):  
        return self.__nombre  
    def verDosis(self):  
        return self.__dosis
```

```
    def asignarNombre(self,med):  
        self.__nombre = med  
    def asignarDosis(self,med):  
        self.__dosis = med
```

```
class Mascota:
```

```
    def __init__(self):  
        self.__nombre=""  
        self.__historia=0  
        self.__tipo=""  
        self.__peso=""  
        self.__fecha_ingreso=""  
        self.__lista_medicamentos=[]
```

```
    def verNombre(self):  
        return self.__nombre  
    def verHistoria(self):  
        return self.__historia  
    def verTipo(self):  
        return self.__tipo  
    def verPeso(self):  
        return self.__peso  
    def verFecha(self):  
        return self.__fecha_ingreso  
    def verLista_Medicamentos(self):  
        return self.__lista_medicamentos
```

```
class sistemaV:
```

```
    def __init__(self):  
        self.__lista_mascotas = []  
        self.__numero_mascotas = len(self.__lista_mascotas)
```





Cuarto ejercicio de abstracción, implementación en python

```
def verificarExiste(self, historia):  
    for m in self.__lista_mascotas:  
        if historia == m.verHistoria():  
            return True  
    #solo luego de haber recorrido todo el ciclo se re  
    return False
```

```
def verNumeroMascotas(self):  
    return len(self.__lista_mascotas)
```

```
def ingresarMascota(self, mascota):  
    self.__lista_mascotas.append(mascota)
```

```
def verFechaIngreso(self, historia):  
    #busco la mascota y devuelvo el atributo solicitado  
    for masc in self.__lista_mascotas:  
        if historia == masc.verHistoria():  
            return masc.verFecha()  
    return None
```

```
def verMedicamento(self, historia):  
    #busco la mascota y devuelvo el atributo solicitado  
    for masc in self.__lista_mascotas:  
        if historia == masc.verHistoria():  
            return masc.verLista_Medicamentos()  
    return None
```

```
def eliminarMascota(self, historia):  
    for masc in self.__lista_mascotas:  
        if historia == masc.verHistoria():  
            self.__lista_mascotas.remove(masc) #opcion con el  
            return True #eliminado con exito  
    return False
```





Cuarto ejercicio de abstracción

Parámetros al ingresar una mascota al servicio:

El servicio de hospitalización solo admite 10 mascotas, se debe avisar desde el principio si hay capacidad para ingresar mascotas nuevas (**Esto debe ir en el menú**)

Antes de solicitar todos los datos de la mascota se debe verificar que no haya otras mascotas con el mismo número de historia clínica (**Esto debe ir en el menú**)

Todos los datos numéricos deben estar validados (**Esto debe ir en el menú**)

Se debe indicar cuántos medicamentos tiene la mascota e ingresar medicamento por medicamento (**En el menú**). En este caso no tendría que verificar que sean medicamentos diferentes

Si se cumplen las condiciones anteriores se agrega la nueva mascota al Sistema (**Esto irá en el método que es el que tiene la lista**)



Cuarto ejercicio de abstracción

```
def main():
    servicio_hospitalario = sistemaV()
    while True:
        menu=int(input('''\nIngrese una opción:
        \n1- Ingresar una mascota
        \n2- Ver fecha de ingreso
        \n3- Ver número de mascotas en el servicio
        \n4- Ver medicamentos que se están administrando
        \n5- Eliminar mascota
        \n6- Salir
        \nUsted ingresó la opción: '''))
        if menu==1: # Ingresar una mascota
            if servicio_hospitalario.verNumeroMascotas() >= 10:
                print("No hay espacio ...")
                continue
            historia=int(input("Ingrese la historia clínica de la mascota: "))
```



Cuarto ejercicio de abstracción

```
historia=int(input("Ingrese la historia clínica de la mascota: "))
#
verificacion=servicio_hospitalario.verDatosPaciente(historia)
if servicio_hospitalario.verificarExiste(historia) == False:

    nombre=input("Ingrese el nombre de la mascota: ")
    tipo=input("Ingrese el tipo de mascota (felino o canino): ")
    peso=int(input("Ingrese el peso de la mascota: "))
    fecha=input("Ingrese la fecha de ingreso (dia/mes/año): ")
    nm=int(input("ingrese la cantidad de medicamento de la mascota: "))
    lista_med=[]

    for i in range(0,nm):
        nombre_medicamentos = input("Ingrese el nombre: ")
        dosis = int(input("Ingrese la dosis: "))
        medicamento = Medicamento()
        medicamento.asignarDosis(dosis)
        medicamento.asignarNombre(nombre_medicamentos)
        lista_med.append(medicamento)
```



Cuarto ejercicio de abstracción

```
mas= Mascota()  
mas.asignarNombre(nombre)  
mas.asignarHistoria(historia)  
mas.asignarPeso(peso)  
mas.asignarTipo(tipo)  
mas.asignarFecha(fecha)  
mas.asignarLista_Medicamentos(lista_med)  
servicio_hospitalario.ingresarMascota(mas)  
# resultado_operacion = servicio_hospitalario.ingresarMascota(mas)  
# if resultado_operacion == False:  
#     print("Capacidad excedida ...")  
else:  
    print("Ya existe una mascota con el numero de historia clínica ingresado.")
```




Cuarto ejercicio de abstracción

Parámetros al ver fecha de ingreso de una mascota:

Se debe solicitar el número de historia clínica, validado, de la mascota **(Esto debe ir en el menú).**

Si la mascota existe se debe imprimir en pantalla la fecha **(El método devuelve la fecha y se imprime en el menú).**

Si la mascota no existe se debe imprimir "La mascota no está en el sistema" **(El método devuelve la cadena con el mensaje y este se imprime en el menú).**



Cuarto ejercicio de abstracción

```
elif menu==2: # Ver fecha de ingreso
    q = int(input("Ingrese la historia clínica de la mascota: "))
    fecha = servicio_hospitalario.verFechaIngreso(q)
    # if servicio_hospitalario.verificarExiste == True
    if fecha != None:
        # fecha = servicio_hospitalario.verFechaIngreso(q)
        # if fecha != None:
        print("La fecha de ingreso de la mascota es: " + fecha)
        # else:
        |    # print("No se asigno fecha")
    else:
        |    print("La historia clínica ingresada no corresponde con ninguna mascota")
```



Cuarto ejercicio de abstracción

Parámetros al ver número de mascotas en el servicio:

Se imprime en pantalla el número de mascotas que haya actualmente en el Sistema **(El método devuelve el número actual de mascotas y este número se imprime en el menú)**

```
elif menu==3: # Ver número de mascotas en el servicio
    numero=servicio_hospitalario.verNumeroMascotas()
    print("El número de pacientes en el sistema es: " + str(numero))
```



Cuarto ejercicio de abstracción

Parámetros al ver medicamentos que se está administrando a una mascota:

Se solicita el número de historia clínica de la mascota **(en el menú)**

Con el número de historia clínica se busca la mascota **(en el método)**

Si la mascota no existe se imprime “La mascota no está registrada en el sistema” **(el método devuelve el mensaje que se imprime en el menú)**

Si la mascota existe se imprime el medicamento **(el método retorna la lista de medicamentos y los nombres de estos se imprimen en el menú)**



Cuarto ejercicio de abstracción

```
elif menu==4: # Ver medicamentos que se están administrando
    q = int(input("Ingrese la historia clínica de la mascota: "))
    medicamento = servicio_hospitalario.verMedicamento(q)
    if medicamento != None:
        print("Los medicamentos suministrados son: ")
        for m in medicamento:
            print(f"\n- {m.verNombre()}")
    else:
        print("La historia clínica ingresada no corresponde con ninguna mascota en el sist
```



Cuarto ejercicio de abstracción

Eliminar una mascota del servicio

Se solicita la historia clínica a eliminar (**en el menú**), si esta historia clínica está en el Sistema se elimina de la lista la mascota asociada

Si la historia clínica no está en el Sistema se envía un mensaje (**desde el método**) que finalmente se imprimirá en el menú principal: La historia clínica no está en el sistema

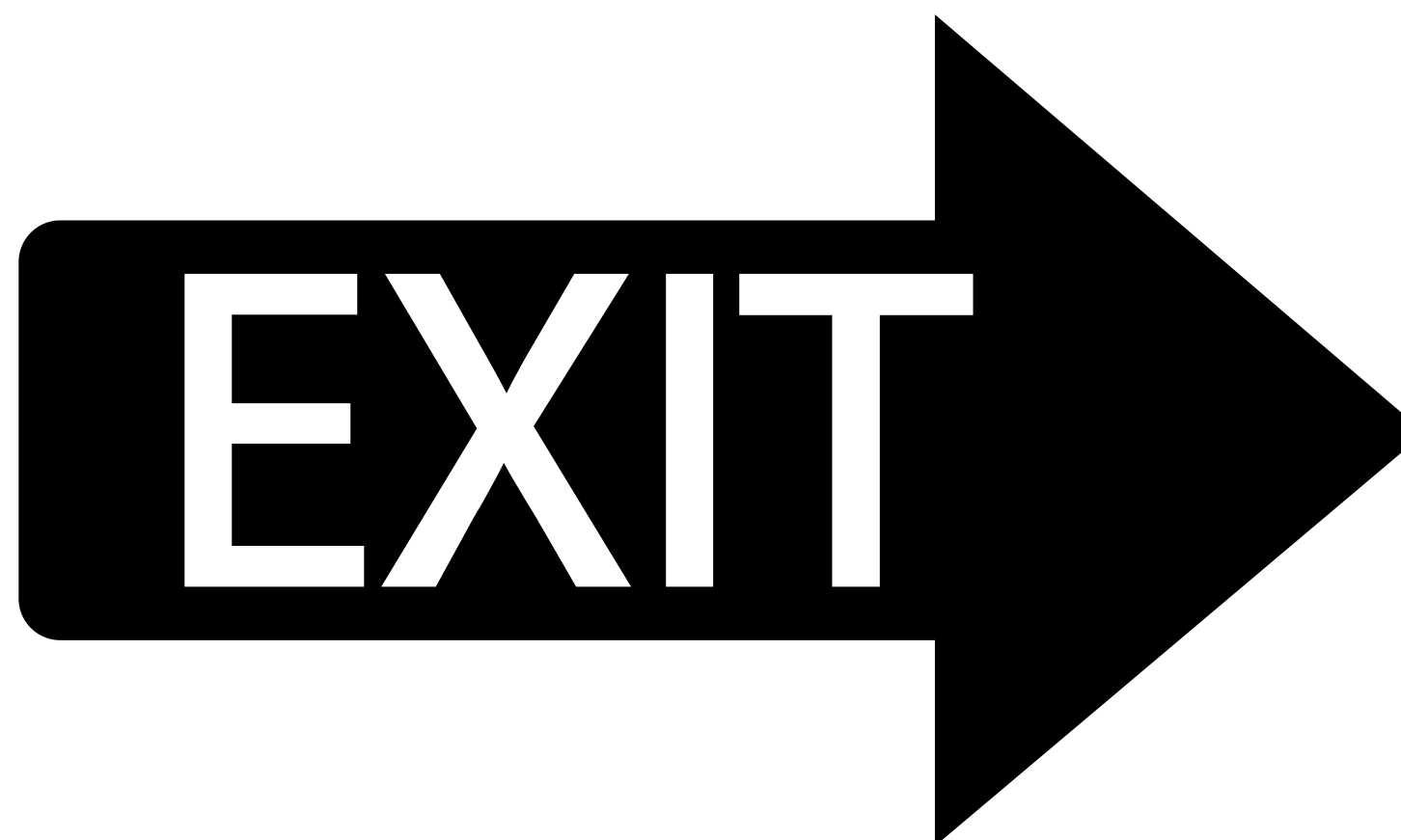
```
elif menu == 5: # Eliminar mascota
    q = int(input("Ingrese la historia clínica de la mascota: "))
    resultado_operacion = servicio_hospitalario.eliminarMascota(q)
    if resultado_operacion == True:
        print("Mascota eliminada del sistema con exito")
    else:
        print("No se ha podido eliminar la mascota")
```



Cuarto ejercicio de abstracción

Parámetros para salir:

Se pide al usuario confirmar
Se Cierra el sistema





UNIVERSIDAD
DE ANTIOQUIA

Facultad de Ingeniería

Problemas adicionales para profundizar



Problemas adicionales para practicar



Modificar el código para que :

- No se pueda ingresar dos medicamentos con el mismo nombre
- 2 elementos contenedores de mascotas (caninos y felinos) dada por diccionarios
- Que la fecha se ingrese como dd/mm/aaaa y que ingresarla de otra manera sea error
- Eliminar medicamento de una mascota

Estudiar el objeto Date de Python y modificar el código para manipular las fechas con este objeto

https://www.w3schools.com/python/python_datetime.asp

GRACIAS

